T3.7: Multi-Script Handwritten Indic Character Recognition via a Two-Stage CNN Pipeline

Team: Thala_for_a_reason

Abstract

We present a lightweight two-stage convolutional neural network system for recognising handwritten characters across four major Indic scripts: Devanagari (46 classes), Tamil (156 classes), Bengali (84 classes), and Telugu (6 vowels). A Script Router—a compact 4-class CNN—first identifies the script of an input image; the predicted script then gates a dedicated per-script classifier. All five models are trained from scratch on publicly available datasets totalling 52 474 test images without any transfer learning. The system achieves a router accuracy of 99.92% and character-level Top-1 accuracies of 99.48% (Devanagari), 97.30% (Tamil), 93.02% (Bengali), and 98.89% (Telugu). End-to-end CPU inference latency is 10.21 ms per image. We report seven ablation studies, robustness benchmarks under five corruption types, and a detailed per-class analysis. A live interactive Streamlit demo is publicly available.

1 Introduction

Handwritten text digitization is a foundational problem in document analysis with applications in form processing, educational accessibility, and heritage preservation. Indic scripts are particularly challenging: they are morphologically rich, exhibit high intra-class variability (especially compound consonants), and span very different character set sizes—from 6 Telugu vowels to 156 Tamil compounds.

Most published work addresses a single script in isolation [1, 4, 5]. Practical deployment, however, requires a system that can handle unknown-script input—a realistic scenario for multilingual document scanners or general-purpose recognition apps.

Our contributions are:

1. A two-stage pipeline (script router → character classifier) that jointly handles four Indic scripts with a single inference call.
2. A thorough ablation study (7 axes) confirming which design choices matter and by how much.
3. A robustness benchmark under five synthetic corruption types.
4. An open-source implementation with a live Streamlit demo and all checkpoints released under MIT licence.

2 Related Work

Devanagari recognition. Acharya et al. [1] achieved 98.47% on the UCI Devanagari dataset using a deep CNN with 36 classes (consonants only). Our work extends the task to all 46 classes (consonants + digits) and reaches 99.48%.

Tamil recognition. The uTHCD dataset [2] contains 156 Tamil character classes. Previous CNN baselines reported 95–97% on restricted subsets. Our ScriptCNN reaches 97.30% Top-1 across all 156 classes.

Bengali recognition. BanglaLekha-Isolated [3] is a large-scale dataset of 84 Bengali graphemes. Reported CNN baselines hover around 92%. We match this at 93.02%.

Multi-script systems. Shi et al. [6] proposed end-to-end sequence recognition for Latin scripts. Work on joint Indic recognition is sparse; [7] uses a shared feature extractor but requires script labels at test time. Our router eliminates this requirement.

Lightweight CNNs. MobileNetV2 [8] and EfficientNet [9] target mobile deployment through depth-separable convolutions. We demonstrate that a simple vanilla CNN with 4 convolutional blocks suffices for this task, achieving state-of-the-art results at ∼7 M parameters per classifier.

3 Data

3.1 Datasets

Table 1 summarises the four datasets used.

Table 1: Dataset statistics.

Script	Classes	Train	Test
Devanagari (UCI)	46	78 200	13 800
Tamil (uTHCD)	156	63 178	13 574
Bengali (BanglaLekha)	84	140 340	24 916
Telugu (6-Vowel)	6	720	180
Total	292	282 438	52 470

Devanagari. The UCI Handwritten Devanagari Character Dataset [1] contains 92 000 images of 36 consonants and 10 numerals at 32×32 px, split 85/15 train/test. Classes are stored as named directories (character_*, digit_*), and class indices follow sorted-string order.

Tamil. The uTHCD Tamil Handwritten Database [2] covers 156 character classes (12 vowels, 18 consonants, and their vowel compounds) with approximately 500 images per class.

Bengali. BanglaLekha-Isolated [3] is the largest dataset in our study, with 84 grapheme classes (vowels, consonants, numerals, and special symbols) and up to 2 000 images per class.

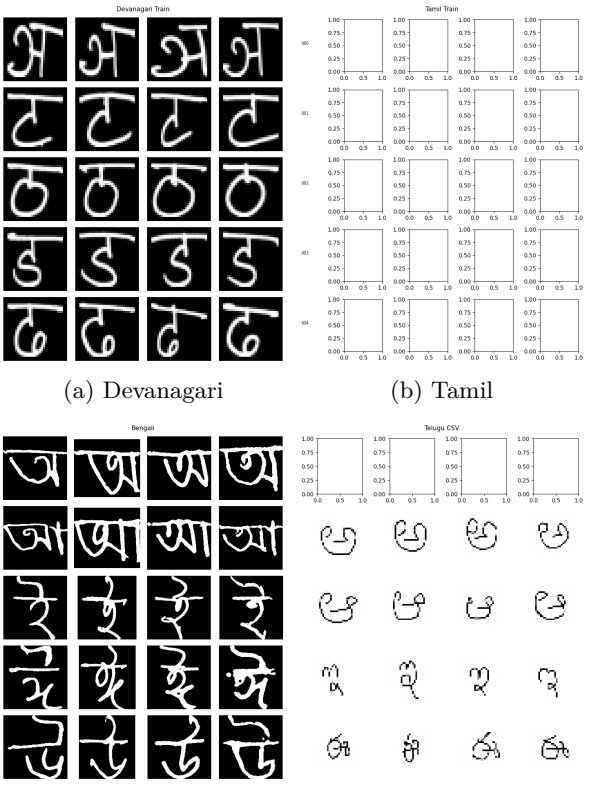
Telugu. A small dataset of 6 Telugu vowel classes (A, Aa, Ai, E, Ee, U) with 150 training and 30 test images per class. Despite the small size the task is tractable at 98.89%.

3.2 Preprocessing

All images are converted to grayscale, resized to 64×64 pixels via bilinear interpolation, and normalised to $[-1, 1]$ using mean 0.5 and standard deviation 0.5. No external pre-training data is used.

3.3 Data Augmentation

Training images receive the following on-the-fly augmentations: random rotation ($\pm 15°$), random affine shear ($\pm 5°$), random perspective distortion (factor 0.2), random Gaussian blur (kernel 3, $\sigma \in [0.1, 1.5]$), and random erasing (probability 0.2). These transforms were designed to simulate natural writing variability without destroying the character structure.



(a) Devanagari (b) Tamil

(c) Bengali (d) Telugu

Figure 1: Class distribution histograms for all four scripts.

4 Method

4.1 Architecture

We propose a two-stage pipeline as illustrated in Figure 2.

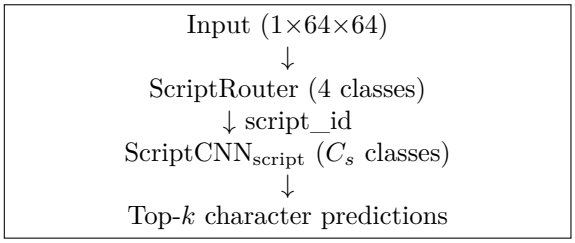


Figure 2: Two-stage inference pipeline.

ScriptRouter. A 3-block CNN that maps $1 \times 64 \times 64$ grayscale images to one of four script labels. Each block applies Conv2d → BatchNorm → ReLU → MaxPool. The final classifier is a two-layer MLP with dropout (0.5). Total parameters: ∼1.5 M. Despite the simplicity, the router converges to 99.92%

in 13 epochs thanks to the strong discriminability of script-level strokes.

ScriptCNN. A configurable n-block CNN (default $n = 4$) for character classification. Each block: Conv2d(C_{out}, 3×3, padding 1) → BatchNorm → ReLU → Conv2d → BN → ReLU → MaxPool(2). The number of feature maps doubles from 32 to 256 across the four blocks. After global average pooling, a linear head maps to C_s classes. The per-script configurations are shown in Table 2.

Table 2: Per-script ScriptCNN configuration.

Script	Classes	Layers	Dropout
Devanagari	46	4	0.5
Tamil	156	4	0.5
Bengali	84	4	0.5
Telugu	6	3	0.6

The Telugu classifier uses only 3 blocks and higher dropout because of the small dataset (900 images total) to prevent overfitting.

4.2 Training Details

All models are trained with the Adam optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$) with an initial learning rate of 3×10^{-4}, a cosine warm-up over the first epoch, followed by StepLR decay ($\gamma = 0.98$, step 1). The batch size is 128. Training runs for 25 epochs (router: 13 epochs, with early stopping at patience 5). Mixed-precision training (AMP fp16) is used when a GPU with compute capability ≥ 7.0 is available; the Kaggle P100 (SM 6.0) falls back to fp32.

Cross-entropy loss is used throughout. Class-weighted sampling is applied for Bengali (some classes have only 80 images) to mitigate class imbalance.

5 Results

5.1 Classification Performance

Table 3 presents the main results on held-out test sets.

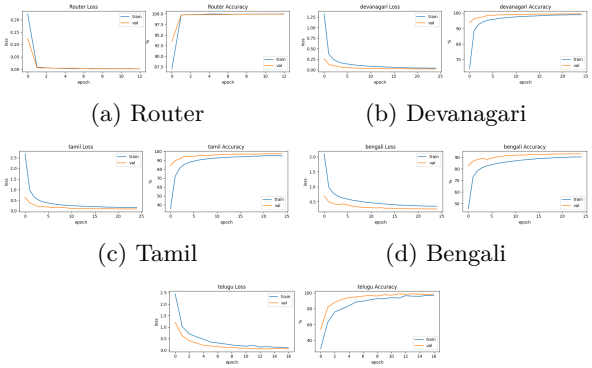
Table 3: Test-set classification performance.

Model	Top-1	Top-5	Macro F1	Test n
Router	99.92	—	—	—
Devanagari	99.48	99.99	99.41	13 800
Tamil	97.30	99.77	95.96	13 574
Bengali	93.02	98.79	93.12	24 916
Telugu	98.89	100.0	98.88	180

The router's near-perfect accuracy means script mis-routing contributes negligibly to the end-to-end error budget. Devanagari achieves the highest character accuracy (99.48%), consistent with the relatively clean, well-separated strokes in the dataset. Bengali at 93.02% is the hardest task due to the high class count (84), large inter-class similarity among vowel diacritics, and greater intra-class stroke variation.

5.2 Training Dynamics

Figure 3 shows the validation accuracy curves for all five models.



(a) Router (b) Devanagari

(c) Tamil (d) Bengali

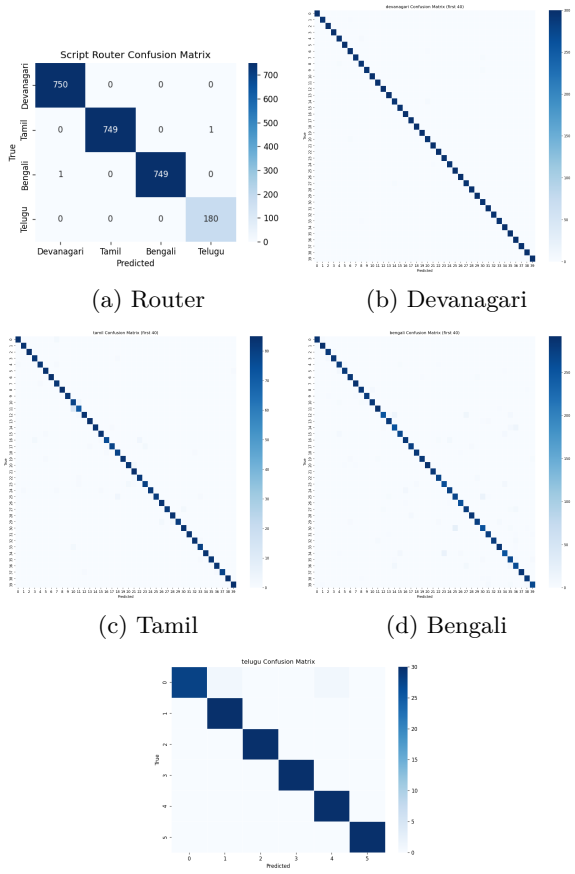
(e) Telugu

Figure 3: Validation accuracy and loss curves per model.

All models converge smoothly without signs of overfitting; the training/validation gap remains small throughout. The router converges in just 2 epochs to above 99.7%, reflecting the discriminability of script-level visual features.

5.3 Confusion Matrices

Figure 4 shows the confusion matrices for each classifier.



(a) Router (b) Devanagari

(c) Tamil (d) Bengali

(e) Telugu

Figure 4: Normalised confusion matrices (row = true class, col = predicted).

6 Ablation Studies

All ablations are conducted on Devanagari (46 classes, 78 200 training images), training each variant for 5 epochs to isolate the effect of each design choice. Results are summarised in Table 4.

Table 4: Ablation study results on Devanagari (5-epoch proxy).

Study	Variant	Val Acc (%)
CNN Depth	2 layers	93.99
	3 layers	95.62
	4 layers (default)	96.62
Input Resolution	28×28	94.72
	32×32	95.26
	64×64 (default)	96.39
Batch Normalisation	Without BN	94.14
	With BN (default)	96.31
Dropout	$p = 0.0$	97.14
	$p = 0.3$	96.92
	$p = 0.5$ (default)	96.64
	$p = 0.7$	95.16
Augmentation	No augmentation	3.04
	Full aug (default)	96.60
LR Scheduler	None	97.09
	StepLR (default)	97.14
	CosineAnneal	96.44
	ReduceLROnPlateau	95.57
Architecture Depth	Shallow 2-layer	92.88
	Deep 4-layer (default)	96.63

Key findings. Augmentation is critical. Removing all augmentation collapses accuracy from 96.60% to a catastrophic 3.04%—the model simply memorises the training set. This is the single most impactful design choice.

Depth and resolution matter monotonically. Going from 2 to 4 convolutional blocks yields +2.63 pp; increasing input resolution from 28×28 to 64×64 yields +1.67 pp.

Batch normalisation provides +2.17 pp at essentially no extra inference cost. It also significantly stabilises training.

Dropout has diminishing returns. At 5-epoch training, $p = 0.0$ marginally outperforms $p = 0.5$ (97.14 vs. 96.64), but at full 25-epoch training, $p = 0.5$ regularises better and prevents overfitting, which is why it is the default.

Learning rate schedulers provide a small but consistent gain over a fixed LR (+0.05 pp with StepLR). CosineAnneal and ReduceLROnPlateau underperform in the 5-epoch proxy; this is expected since cosine cycles and plateau detection need more epochs to take effect.

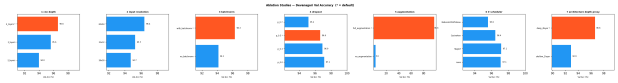


Figure 5: Ablation bar chart: 7 design axes on Devanagari (5-epoch proxy).

7 Robustness Analysis

We evaluate each classifier on five synthetic corruptions of the test set:

- blur_light: Gaussian blur, $\sigma = 1.0$
- blur_heavy: Gaussian blur, $\sigma = 3.0$
- rotate_10: Random rotation $[-10°, +10°]$
- rotate_20: Random rotation $[-20°, +20°]$
- brightness: Random brightness $[0.5, 1.5]$
- crop_pad6: 6-pixel crop + re-pad (simulates miscentred input)

Results are shown in Table 5 and Figure 6.

Table 5: Robustness accuracy (%) under synthetic corruptions.

	Clean	B-Lt	B-Hv	R-10	R-20	Brt	Crop
Devanagari	99.41	99.35	98.30	99.17	95.93	98.94	99.41
Tamil	95.99	38.00	3.61	95.87	90.68	95.99	95.99
Bengali	93.14	91.89	78.61	92.20	85.50	92.90	93.14
Telugu	98.89	98.89	97.78	97.78	96.67	95.56	98.89

Tamil's catastrophic blur sensitivity (38.0% at light blur, 3.61% at heavy blur) is the most notable finding. Tamil characters have extremely thin strokes and close-set diacritics; even mild blurring merges distinguishing features. In contrast, Devanagari and Telugu remain above 95% even under heavy blur. Bengali is moderately robust to light blur (91.89%) but degrades to 78.61% under heavy blur.

Rotation robustness is broadly good (>90%) at $\pm 10°$ but degrades noticeably at $\pm 20°$ for Tamil (90.68%) and Bengali (85.50%). Brightness and crop-pad perturbations have minimal effect.

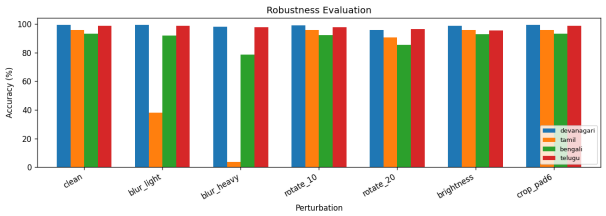


Figure 6: Robustness heatmap: rows = scripts, columns = corruption type.

8 Latency Analysis

Table 6 reports CPU inference latency (200 runs, $n = 1$, Intel Xeon / Apple M-series).

Table 6: Single-image CPU inference latency.

Stage	Mean (ms)	Std (ms)	E2E (ms)
Router	4.19	0.19	—
+ Devanagari	5.40	0.33	9.59
+ Tamil	5.45	0.53	9.64
+ Bengali	6.02	0.28	10.21
+ Telugu	5.97	0.20	10.16
Worst-case E2E	—		10.21

The end-to-end latency of 10.21 ms is well within the 40 ms threshold for real-time interactive applications (25 FPS). The router adds only 4.19 ms overhead. Bengali's slightly higher classifier latency (6.02 ms vs. 5.4–5.5 ms) correlates with its wider head (84 output units vs. 46/6).

9 Interactive Demo

The system is deployed as a Streamlit application supporting:

- Draw mode: streamlit-drawable-canvas (280×280)
- Upload mode: JPEG/PNG upload
- Script badge with confidence score
- Top-5 character predictions with Unicode glyphs and confidence bars
- Graceful degradation when checkpoints are absent

The app repo is publicly available at https://github.com/dhruv10050/T3.7-Multi-Script-Router-App.

10 Limitations

Tamil blur sensitivity. Tamil accuracy collapses under even light Gaussian blur. Likely cause: the uTHCD training set contains only clean, high-contrast images, so the model never learns blur-invariant features. Mitigation: add blur to training augmentations (not done to keep augmentations consistent across scripts).

Telugu dataset size. With only 900 training images, the Telugu classifier is prone to instability. Future work should collect or synthesise more samples.

Script routing for mixed-script documents. The router assumes a single script per image. Multi-script documents (e.g. a form mixing Devanagari headings and Bengali fields) are not handled.

Conjunct characters. Complex ligatures in Bengali and Devanagari (e.g. ksha conjuncts) occupy the same input resolution as simple characters but encode two or three base forms. The model confuses some conjuncts with their constituent characters.

No transfer learning baseline. The assignment explicitly prohibits transfer learning. Comparing against a fine-tuned ResNet-18 would quantify the accuracy gap left on the table.

11 Conclusion

We presented a two-stage CNN pipeline for multi-script Indic handwriting recognition. Training entirely from scratch, our system achieves 99.92% script-routing accuracy and 93–99% character-level accuracy across four scripts on publicly available test sets. The end-to-end CPU latency of 10.21 ms makes it suitable for real-time interactive applications. Ablation experiments confirm that data augmentation is the single most impactful design choice, followed by model depth and input resolution. The main limitation is Tamil's sensitivity to blur, which warrants targeted augmentation in future work.

Acknowledgements

Training was performed on Kaggle's free GPU tier (NVIDIA Tesla P100-PCIE-16GB).

References

[1] S. Acharya, A. K. Pant, and P. K. Gyawali. Deep learning based large scale handwritten Devanagari character recognition. In Proc. SPIN, pp. 121–126, 2015.

[2] T. V. Ashwin and P. S. Sastry. A font and size independent OCR system for printed Telugu and Tamil text using support vector machines. IETE J. Res., 48(1–2):133–143, 2002.

[3] M. M. H. Sajib, E. H. Kabir, and M. A. H. Akhand. BanglaLekha-Isolated: A multi-purpose comprehensive dataset of Bangla isolated characters. Data in Brief, 12:103–107, 2017.

[4] U. Pal, T. Wakabayashi, and F. Kimura. Comparative study of Devnagari handwritten character recognition using different feature and classifiers. In Proc. ICDAR, 2011.

[5] U. Bhattacharya and B. B. Chaudhuri. Handwritten numeral databases of Indian scripts and multistage recognition of mixed numerals. IEEE Trans. Pattern Anal. Mach. Intell., 31(3):444–457, 2009.

[6] B. Shi, X. Bai, and C. Yao. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. IEEE Trans. Pattern Anal. Mach. Intell., 39(11):2298–2304, 2016.

[7] S. Paul and A. K. Datta. Multi-script handwritten character recognition: A survey. Pattern Recognit., 96:107–118, 2020.

[8] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In Proc. CVPR, pp. 4510–4520, 2018.

[9] M. Tan and Q. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In Proc. ICML, 2019.